

# **UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN**

---

## **TECNOLOGÍAS DE LA INFORMACIÓN DESARROLLO DE SOFTWARE**

### **EJEMPLO INTEGRADOR S10**

#### **LOGIN SEGURO, ACCESIBLE Y CONTROLADO POR TECLADO**

**PRESENTA:  
OSCAR ESPINOZA LANDETA  
8 "A"**

**A CARGO:  
ING. JOSÉ MIGUEL CARRERA PACHECO**



# Investigación Técnica: Implementación de Sistemas de Autenticación Modernos, Seguros y Accesibles

## 1. Comportamiento de un formulario accesible

Un formulario accesible es aquel que permite a cualquier usuario, independientemente de sus capacidades visuales, motoras o cognitivas, percibir, entender, navegar e interactuar con éxito. Para lograrlo, el desarrollo debe apegarse a las Pautas de Accesibilidad para el Contenido Web (WCAG), garantizando que la estructura semántica y el comportamiento dinámico sean predecibles.

### Principios de accesibilidad aplicados a formularios

- **Perceptible:** Los campos y mensajes deben ser identificables (ej. buen contraste, etiquetas visibles y legibles por software).
- **Operable:** Todo el formulario debe ser navegable y enviables utilizando únicamente el teclado.
- **Comprensible:** Las instrucciones, los formatos esperados y los errores deben ser claros y directos.
- **Robusto:** Debe ser compatible con tecnologías de asistencia, como lectores de pantalla (NVDA, JAWS, VoiceOver).

### Uso correcto de etiquetas y atributos ARIA

HTML semántico es la base, pero los atributos ARIA (Accessible Rich Internet Applications) proveen contexto adicional cuando el HTML nativo se queda corto, especialmente en estados dinámicos.

- **<label>:** Todo campo de entrada (**<input>**) debe tener una etiqueta **<label>** asociada explícitamente mediante el atributo **for** (que coincida con el **id** del input). Esto amplía el área de clic y da nombre accesible al campo.
- **aria-invalid:** Indica si el valor ingresado en un campo tiene un error. Toma los valores **true** o **false**. Cuando un lector de pantalla enfoca un input con **aria-invalid="true"**, anuncia "entrada no válida", alertando al usuario inmediatamente.
- **aria-describedby:** Vincula un campo de entrada con un texto descriptivo adicional (como instrucciones o mensajes de error) mediante el **id** del elemento de texto. Cuando el usuario enfoca el campo, el lector de pantalla lee la etiqueta y, a continuación, la descripción vinculada.
- **aria-live:** Convierte un contenedor en una "región viva". Se utiliza para inyectar mensajes de error globales. Un valor **aria-live="polite"** anuncia el cambio cuando el lector de pantalla termina su tarea actual; **aria-live="assertive"** interrumpe al lector para anunciar errores críticos.

### Interacción con lectores de pantalla y navegación por teclado

Un lector de pantalla procesa un login leyendo secuencialmente el título, las instrucciones y los campos. Al llegar a un input, anuncia su tipo (ej. "Contraseña, edición protegida"), su etiqueta y si es obligatorio.

Para la navegación por teclado:

- **Tab / Shift+Tab:** Mueve el foco hacia adelante o hacia atrás a través de elementos interactivos (inputs, botones, enlaces) en un orden lógico (de arriba hacia abajo, de izquierda a derecha).
- **Enter:** Estando el foco dentro de cualquier campo de texto, presionar "Enter" debe enviar el formulario automáticamente.
- **Espacio/Enter:** Activa botones (como "Mostrar contraseña").

---

## 2. Errores comunes en formularios de login

El diseño deficiente de un login introduce brechas críticas tanto de seguridad como de accesibilidad.

### Errores frecuentes y su impacto

1. **Filtración de información en mensajes de error:**
    - *Error:* Mostrar "El usuario no existe" o "Contraseña incorrecta".
    - *Riesgo:* Permite a un atacante realizar "User Enumeration" (enumeración de usuarios). Si el sistema confirma qué usuarios existen, el atacante solo necesita usar fuerza bruta contra las contraseñas de esos usuarios válidos.
  2. **Validaciones que rompen la UX (Validación prematura):**
    - *Error:* Mostrar un error de "Formato de correo inválido" en el momento en que el usuario digita la primera letra de su email.
    - *Riesgo:* Genera frustración cognitiva. La validación debe ocurrir idealmente cuando el usuario abandona el campo (**onBlur**) o al intentar enviar el formulario.
  3. **Dependencia exclusiva del color para los errores:**
    - *Error:* Indicar un error únicamente poniendo el borde del input en rojo.
    - *Riesgo:* Los usuarios con daltonismo no percibirán el error, incumpliendo las normas WCAG. Siempre debe acompañarse de un icono y texto descriptivo.
  4. **Uso de *Placeholders* como etiquetas:**
    - *Error:* Omitir el `<label>` y usar el atributo **placeholder** para indicar qué se debe ingresar.
    - *Riesgo:* Al escribir, el placeholder desaparece, causando pérdida de contexto. Además, muchos lectores de pantalla no leen los placeholders consistentemente, y su contraste suele ser deficiente.
  5. **Trampas de teclado y falta de *Focus Visibility*:**
    - *Error:* Elementos que capturan el foco (Tab) y no permiten salir con teclado, o la eliminación del contorno de foco (**outline: none** en CSS sin un reemplazo).
    - *Riesgo:* Un usuario que depende del teclado no sabrá en qué parte del formulario se encuentra, haciendo imposible el inicio de sesión.
-

### 3. Manejo seguro de sesiones

El protocolo HTTP carece de estado (stateless). Una sesión de usuario es el mecanismo que permite a la aplicación recordar que un usuario específico se ha autenticado con éxito en solicitudes subsecuentes.

#### Estrategias de sesión

- **Sesiones tradicionales (Stateful):** El servidor crea un registro de sesión en su base de datos o memoria y envía al cliente un identificador único (Session ID) almacenado en una cookie. El servidor debe validar este ID en cada petición.
- **JWT (JSON Web Tokens - Stateless):** El servidor firma criptográficamente un token que contiene la información del usuario (claims) y lo envía al cliente. El servidor no guarda la sesión; solo verifica la firma del token en cada petición.
- **Cookies seguras:** Son el vector de transporte ideal para identificadores de sesión o tokens, configuradas para mitigar ataques.

#### Buenas prácticas de seguridad en Cookies

Para proteger la sesión, las cookies deben configurarse con las siguientes directivas:

- **HttpOnly:** Impide que lenguajes de scripting del lado del cliente (JavaScript) accedan a la cookie. Mitiga ataques de Cross-Site Scripting (XSS).
- **Secure:** Garantiza que la cookie solo se transmita a través de conexiones encriptadas (HTTPS), previniendo la interceptación en redes no seguras (Man-in-the-Middle).
- **SameSite (Strict o Lax):** Controla si la cookie se envía en solicitudes de origen cruzado, protegiendo contra ataques de falsificación de solicitudes entre sitios (CSRF).
- **Expiración y Regeneración:** Las sesiones deben tener un tiempo de vida limitado (TTL). Además, al autenticarse, el Session ID debe regenerarse para evitar ataques de fijación.

#### Riesgos comunes

- **Session Hijacking (Secuestro de sesión):** Un atacante roba el token de sesión válido de un usuario (vía XSS o redes no seguras) y se hace pasar por él.
- **Session Fixation (Fijación de sesión):** El atacante obliga a la víctima a usar un Session ID que el atacante ya conoce (por ejemplo, enviándole un enlace con el ID en la URL). Si el sistema no regenera el ID tras el login, el atacante mantendrá el acceso.

---

### 4. Manejo seguro de errores de autenticación

La forma en que el sistema comunica los fallos es una línea de defensa vital.

#### Principio de error genérico

La regla de oro de la autenticación es no dar pistas al atacante.

- **Incorrecto:** "El correo ingresado no está registrado."
- **Correcto:** "El correo electrónico o la contraseña son incorrectos."

## Manejo de intentos fallidos

Un sistema robusto debe asumir que estará sujeto a ataques de diccionario o fuerza bruta.

- **Rate Limiting (Limitación de tasa):** Restringir el número de peticiones de login desde una misma dirección IP en un periodo determinado (ej. máximo 5 intentos por minuto).
- **Bloqueo temporal (Account Lockout):** Tras \$N\$ intentos fallidos consecutivos contra una misma cuenta, bloquear temporalmente la capacidad de intentar nuevamente (ej. bloqueo progresivo de 5, 15 y 60 minutos).
- **Logs de seguridad:** Registrar todos los intentos fallidos, incluyendo marca de tiempo, dirección IP y el correo intentado (sin registrar nunca la contraseña). Esto permite auditorías y alertas automatizadas ante comportamientos anómalos.

---

## 5. Diseño de un login accesible y profesional

Un diseño óptimo equilibra la fricción de seguridad con la fluidez de UX.

- **Claridad visual y jerarquía:** El formulario debe estar centrado o destacado. Los campos deben tener un tamaño adecuado para interactuar en pantallas táctiles (mínimo 44x44 CSS pixels para botones).
- **Feedback claro:** Proporcionar un botón para "Mostrar/Ocultar contraseña". Este botón debe ser accesible por teclado y su estado (**aria-pressed** o cambio de etiqueta "Ocultar") debe ser anunciado por el lector de pantalla.
- **Estados de carga (Loading):** Al enviar el formulario, el botón debe desactivarse (**disabled**) para evitar múltiples envíos accidentales, y proporcionar feedback visual (un spinner) y auditivo (**aria-live** anunciando "Iniciando sesión...").
- **Recuperación de contraseña:** Siempre debe existir un flujo claro y accesible para recuperar credenciales, situado cerca del campo de contraseña o botón de envío.

---

## 6. Implementación práctica

A continuación se detalla el flujo completo de una implementación moderna.

### Flujo de Autenticación

1. **Envío:** El usuario llena los campos. El frontend valida que los campos no estén vacíos y que el correo tenga formato válido (añadiendo **aria-invalid="true"** dinámicamente si hay error local).
2. **Transmisión:** Se envía la petición por HTTPS al endpoint del backend (ej. **POST /api/auth/login**).
3. **Validación Backend:** El servidor sanitiza los inputs, busca al usuario en la base de datos y compara el hash de la contraseña usando algoritmos seguros (como Argon2 o bcrypt).

4. **Gestión de Sesión:** Si es exitoso, el servidor regenera la sesión, crea un JWT o Session ID, y lo inyecta en la respuesta como una cookie **HttpOnly**, **Secure** y **SameSite**.
5. **Redirección:** El servidor responde con el estado 200 OK y los datos del rol del usuario. El frontend redirige al usuario al panel correspondiente (ej. **/admin-dashboard** o **/user-profile**).

## Estructura HTML Accesible (Ejemplo)

HTML

```
<form novalidate aria-describedby="form-error">
  <div id="form-error" aria-live="assertive" class="error-message"></div>

  <div class="form-group">
    <label for="email">Correo electrónico</label>
    <input type="email" id="email" name="email" required
      aria-describedby="email-error" aria-invalid="false">
    <span id="email-error" class="error-text"></span>
  </div>

  <div class="form-group">
    <label for="password">Contraseña</label>
    <div class="password-wrapper">
      <input type="password" id="password" name="password" required
        aria-describedby="password-error" aria-invalid="false">
      <button type="button" aria-label="Mostrar contraseña">👁️</button>
    </div>
    <span id="password-error" class="error-text"></span>
  </div>

  <button type="submit">Iniciar Sesión</button>
</form>
```

---

## 7. Testing del sistema de login

Un sistema crítico requiere una estrategia de pruebas multicapa para certificar que cumple con los requerimientos técnicos.

- **Pruebas unitarias (Backend):** \* Verificar que la función de *hashing* genere hashes únicos y consistentes.
  - Probar la lógica de validación de roles y la correcta emisión de tokens JWT.
- **Pruebas de seguridad (Penetration Testing):**
  - Intentos de inyección SQL en los campos del formulario.
  - Pruebas de fuerza bruta para validar que el *Rate Limiting* y el bloqueo de cuentas funcionen correctamente.
  - Inspección de las cabeceras HTTP de respuesta para confirmar la presencia de flags en las cookies (**HttpOnly**, **Secure**).
- **Pruebas de validación de formularios:**

- Ingresar cadenas maliciosas (ej. etiquetas `<script>`) para confirmar sanitización y evasión de XSS.
- Evaluar casos límite (correos con formatos inusuales permitidos por RFC, contraseñas extremadamente largas).
- **Pruebas de accesibilidad automatizadas:**
  - Ejecutar herramientas como **Lighthouse** o **axe-core** en el pipeline CI/CD para detectar errores de contraste, etiquetas faltantes o jerarquía incorrecta.
- **Pruebas de navegación manuales:**
  - **Prueba de teclado:** Desconectar el ratón y asegurar que es posible navegar, rellenar y enviar el formulario fluidamente usando solo Tab, Shift+Tab y Enter, verificando que el indicador de foco sea siempre visible.
  - **Prueba de lector de pantalla:** Utilizar NVDA o VoiceOver para recorrer el formulario, asegurándose de que los estados de error inyectados por JavaScript (`aria-invalid`, `aria-live`) se anuncien correctamente en tiempo real.